



Grado Ingeniería Electrónica Industrial y Automática

Informática Industrial I

Miguel Lansac Hernanz- 100471968

Rafael Torres Olmedo- 100471922

Índice

1. Introducción.
2. diagramas UML.
 - 2.1. Diagrama pre-trabajo.
 - 2.2. Diagrama post-trabajo.
3. Explicación del programa.
 - 3.1. Herencia:
 - 3.2. Constructores y destructores:
 - 3.3. Métodos Get Set:
 - 3.4. Métodos de visualización:
 - 3.5. Inserción/borrado/edición de usuarios y eventos:
 - 3.6. Uso efectivo de contenedores (vector<> o arrays dinámicos):
 - 3.7. Sistema de compra/venta de entradas y su gestión:
 - 3.8. Almacenamiento en ficheros:
 - 3.9. Menús e interfaz:
 - 3.10. Sobrecarga de operadores
4. Optimización y conclusión.

1. Introducción.

Tras haber leído el enunciado e ido a las prácticas nosotros optamos por un programa en el que hubiese una clase común para casi todas las clases, Esta se llamará Usuario, de la que heredan varias clases, así consiguiendo también la clase virtual , que es uno de los requerimientos.

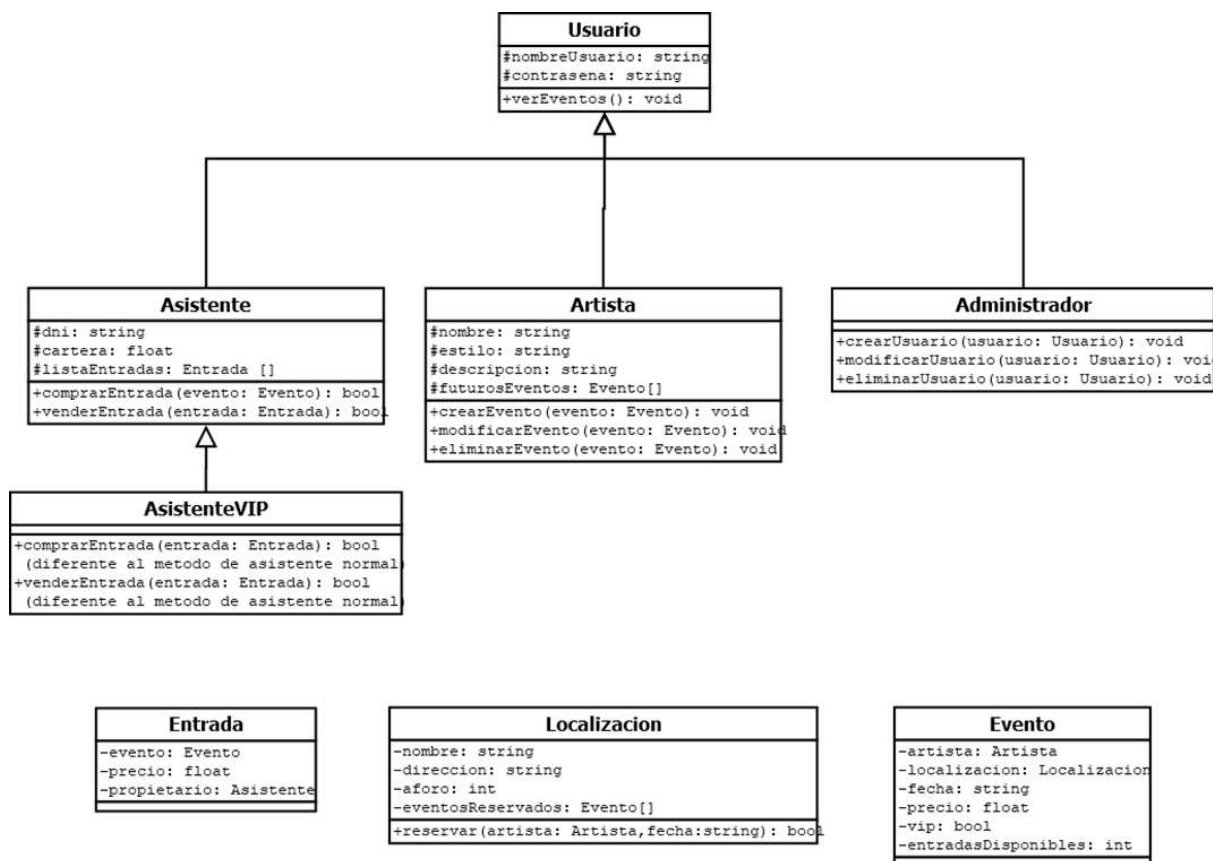
El presente trabajo tiene como finalidad el desarrollo de una aplicación orientada a la gestión digital de entradas para conciertos, implementada en lenguaje C++. Esta plataforma permite tanto la compra como la venta de entradas por parte de los distintos tipos de usuarios que interactúan en el sistema, siguiendo los principios fundamentales de la programación orientada a objetos.

A lo largo del proyecto se ha diseñado e implementado un sistema que simula una plataforma real de gestión de eventos musicales, contemplando las necesidades específicas de tres tipos de usuarios: artistas, asistentes (incluyendo asistentes VIP) y administradores. Cada uno de estos perfiles cuenta con funcionalidades diferenciadas que reflejan su papel dentro del sistema, aplicando conceptos de herencia, polimorfismo y encapsulamiento, pilares fundamentales del paradigma objeto.

El trabajo ha tenido una evolución en cada entrega, es decir, se ha pasado de cosas básicas como imprimir por pantalla un artista en la primera entrega, a poder eliminar y modificar eventos y usuarios, sin importar el tipo de usuario (artista, asistente etc...), comprar y vender entradas y el volcado de ficheros.

2. Diagramas UML.

2.1. Diagrama pre-trabajo.

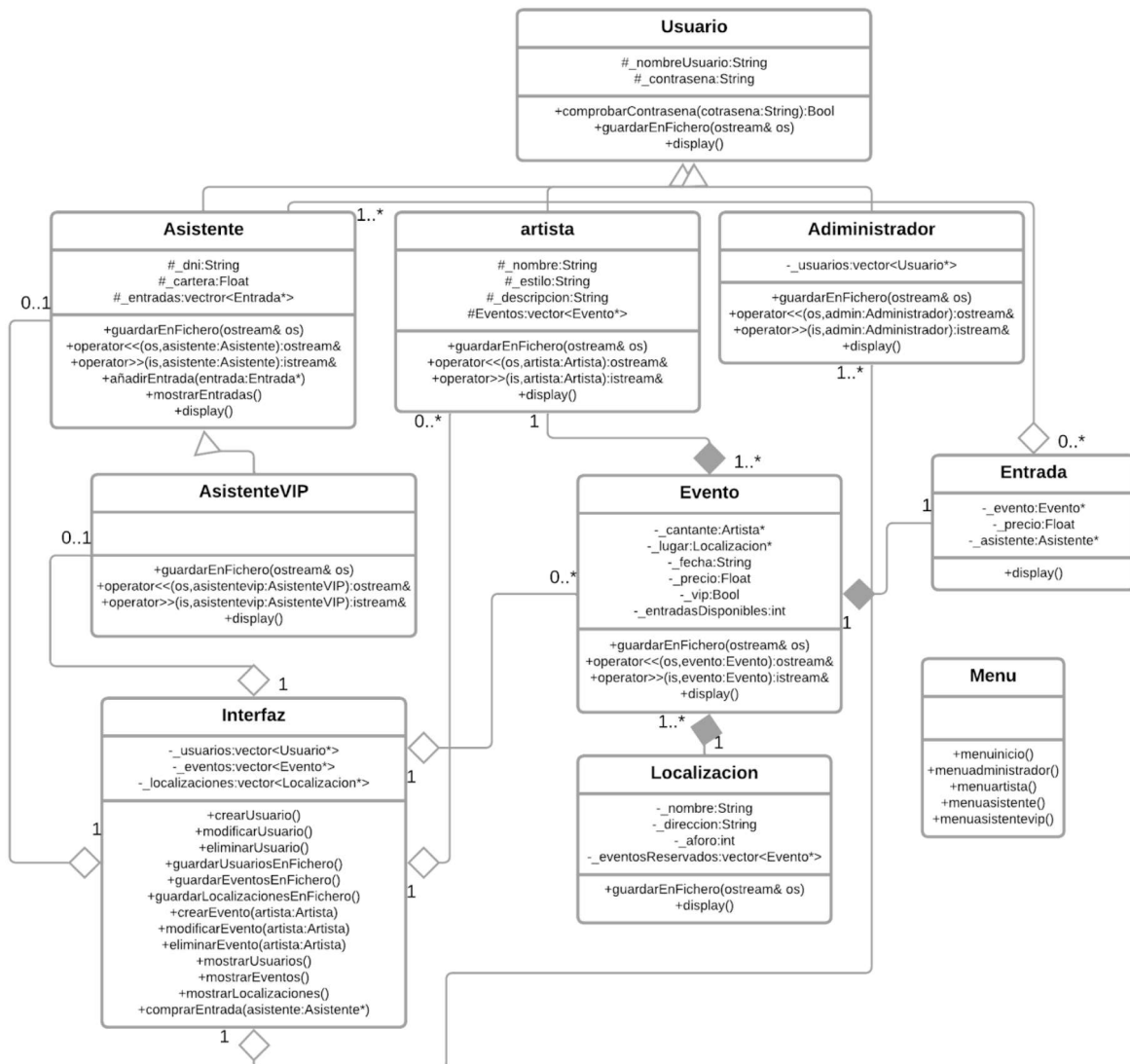


Este es el diagrama UML que hemos propuesto, en el que usuario es la clase padre de Asistente, Artista y Administrador y a su vez asistente es la clase padre de Asistente VIP. También tenemos las clases Entrada, Localización y Evento.

Las entradas pertenecen a un evento. Evento tiene una relación con Localización en la que para cada evento habrá que elegir una localización. Los asistentes compran y venden entradas.

Además de los métodos puestos en el diagrama UML cada clase tendrá sus constructores y destructor y cada atributo tendrá su setter y getter.

2.2. Diagrama post-trabajo.



Este sería el segundo diagrama UML, este será el diagrama final, con todas las funciones necesarias para que funcione el programa. Como se puede apreciar, hay un gran cambio con el primer diagrama UML, principalmente se puede señalar la aparición de nuevas clases, como **Menu**, que se encarga de imprimir los distintos menús que luego implementamos en el main para que la persona que está compilando el proyecto pueda elegir entre las distintas opciones, que luego un switch se encargará de, dependiendo qué opción se escoja, hacer una cosa u otra. La otra clase nueva sería **Interfaz**, que se encargará de todo el tema de eliminar usuarios, crearlos, eliminar eventos, crearlos etc....

Hemos decidido implementar estas clases a nuestro proyecto, porque nos permiten acceder desde distintos puntos a funciones que vamos a utilizar en distintas clases,

o incluso en el main, de esta forma optimizamos el código. También podemos apreciar algunas funciones distintas, como pueden ser mostrarusuario() que es una función virtual de la clase usuario, que es una clase virtual, ya que esa función la usaran artistas Empleados Asistentes y AsistentesVIP, y lo que hará es enseñar por pantalla las distintas clases con sus atributos.

También cabe destacar la sobrecarga de operadores (>>) que la añadimos para esta última función y para mostrarEventos() y mostrarLocalizaciones(), de esta forma nos ahorramos bastantes líneas de código, ya que no hay que estar escribiendo los "cout<<get...()" cada vez que queramos imprimir por pantalla cada clase.

En este UML no hemos añadido los getters y los setters debido a que quedarían unas clases muy grandes y no se entenderían nada las conexiones, al igual que con los constructores y destructores.

3. Explicación del programa.

3.1 Herencia y polimorfismo:

La herencia de este proyecto está basada en los usuarios, con una clase padre llamada Usuario y sus hijas que heredan de ella Artista, Administrador y Asistente que a su vez es padre de AsistenVIP.

Usuario tiene dos métodos virtuales puros:

```
19     virtual void guardarEnFichero(ostream& os) const = 0; //metodo virtual puro
20     virtual void display() const = 0; //metodo virtual puro
```

Los cuales también están presentes en las clases hijas, aunque en cada una de ellas hagan cosas diferentes. Que estos métodos sean virtuales puros hace que la clase usuario sea una clase abstracta.

En cuanto al polimorfismo, este lo podemos ver en momentos como este:

```
void Interfaz::mostrarUsuarios() const
{
    cout << " ===== Lista de usuarios ===== " << endl;

    for (int i = 0; i < _usuarios.size(); i++)
    {
        cout << i + 1 << ". ";
        _usuarios[i]->display(); // aqui hay polimorfismo porque desde usuario llamamos a las de las hijas
    }
    cout << " ===== FIN Lista de usuarios ===== " << endl;
}
```

Donde desde usuarios llamamos a la función display aunque esta sea ejecutada por las hijas dependiendo de el usuario que sea cada miembro del vector _usuarios.

3.2 Constructores y destructores:

Para todas las clases hemos creado sus respectivos constructores y destructores, para todas hemos creado un constructor vacío, en el que inicializamos todos los atributos de la clase a "0" y un constructor parametrizado para poder crear objetos de esas clases tan solo pasando los datos que quisiéramos, algo muy útil que

hemos usado en el resto de código. Y por último hemos creado los destructores que aunque en la mayoría están vacíos, cuando hemos usado memoria dinámica estos los hemos usado para destruir dicha memoria, como se adjunto en la siguiente imagen.

```
7  ▾ Interfaz::Interfaz(vector<Usuario*> usuarios, vector<Localizacion*> localizaciones, vector<Evento*> eventos)
8  {
9      _usuarios = usuarios;
10     _localizaciones = localizaciones;
11     _eventos = eventos;
12 }
13 ▾ Interfaz::~Interfaz() //como se crean con new hay que borrarlos despues
14 {
15     for (int i = 0; i < _usuarios.size(); i++)
16     {
17         delete _usuarios[i];
18     }
19
20     for (int i = 0; i < _localizaciones.size(); i++)
21     {
22         delete _localizaciones[i];
23     }
24
25     for (int i = 0; i < _eventos.size(); i++)
26     {
27         delete _eventos[i];
28     }
29 }
30
```

3.3 Getters y Setters:

Para todas las funciones hemos usado métodos getters y setters que hacen su respectiva tarea de devolver o asignar el atributo seleccionado, para los getters, los hemos declarado como constantes, ya que estos no modifican los atributos. Usamos getters y setters en todos los atributos de todas las clases ya que hemos puesto privados o protegidos estos, asique a la hora de acceder a ellos era imposible sin los getters y setters.

```
21 ▾ string Artista::getnombre() const
22 {
23     return _nombre;
24 }
25 ▾ void Artista::setnombre(string nombre)
26 {
27     _nombre=nombre;
28 }
29 ▾ string Artista::getEstilo() const
30 {
31     return _estilo;
32 }
33 ▾ void Artista::setEstilo(string estilo)
34 {
35     _estilo=estilo;
36 }
37 ▾ string Artista::getDescripcion() const
38 {
39     return _descripcion;
40 }
41 ▾ void Artista::setDescripcion(string descripcion)
42 {
43     _descripcion=descripcion;
44 }
```

Este es un ejemplo de la clase artista.

3.4 Métodos de visualización:

Como métodos de visualización, hemos usado en interfaz mostrarUsuarios, mostrarEventos y mostrarLocalizaciones, que recorrian los vectores de cada tipo y llamaban al método display de cada tipo, en usuarios llamaban al display de cada hija, en eventos llamaban al display de eventos y en localizaciones al de localizaciones. También hemos creado métodos display para ver las entradas en la clase asistente.

```
406 void Interfaz::mostrarUsuarios() const
407 {
408     cout << " ===== Lista de usuarios ===== " << endl;
409
410     for (int i = 0; i < _usuarios.size(); i++)
411     {
412         cout << i + 1 << ". ";
413         _usuarios[i]->display(); // aqui hay polimorfismo porque desde usuario llamamos a las de las hijas
414     }
415     cout << " ===== FIN Lista de usuarios ===== " << endl;
416 }
417 void Interfaz::mostrarEventos() const
418 {
419     cout << " ===== Lista de eventos ===== " << endl;
420
421     for (int i = 0; i < _eventos.size(); i++)
422     {
423         cout << i + 1 << ". ";
424         _eventos[i]->display(); // aqui hay polimorfismo porque desde usuario llamamos a las de las hijas
425     }
426     cout << " ===== FIN Lista de eventos ===== " << endl;
427 }
428 void Interfaz::mostrarLocalizaciones() const
429 {
430     cout << " ===== Lista de localizaciones ===== " << endl;
431
432     for (int i = 0; i < _localizaciones.size(); i++)
433     {
434         cout << i + 1 << ". ";
435         _localizaciones[i]->display(); // aqui hay polimorfismo porque desde usuario llamamos a las de las hijas
436     }
437     cout << " ===== FIN Lista de localizaciones ===== " << endl;
438 }
439
440
441
442
443
444
445
446
447 void Administrador::display() const
448 {
449     cout << "-Tipo: Administrador " << endl;
450     cout << " -Nombre de usuario: " << getnombreUsuario() << endl;
451     cout << " -Contraseña: " << getcontrasena() << endl;
452     cout << " ===== " << endl;
453
454 }
```

De esta manera logramos mostrar por pantalla todos los datos.

3.5 Inserción/borrado/edición de usuarios y eventos:

Para la creación, modificación y eliminación de eventos y usuarios hemos creado seis funciones en la clase interfaz, en las que para las acciones de los usuarios no recibes ni devuelves nada, mientras que para los métodos de los eventos enviamos un puntero al artista, éste artista será el que esté manejando el programa en el momento y solo podrá crear, modificar o eliminar eventos propios.

```

26 // Métodos para usuarios
27 void crearUsuario();
28 void modificarUsuario();
29 void eliminarUsuario();

38 // Métodos para eventos
39 void crearEvento(Artista* artista);
40 void modificarEvento(Artista* artista);
41 void eliminarEvento(Artista* artista);

```

3.6 Uso efectivo de contenedores (vector<> o arrays dinámicos):

Para la implementación de listas de usuarios, eventos y localizaciones hemos creado tres vectores privados en la clase interfaz la cual maneja todos los vectores. Para los usuarios aunque sean de diferentes tipos los guardamos todos en el mismo vector de la clase padre y luego usaremos el “dynamic_cast” para saber de qué clase hija son.

```

56 private:
57     vector<Usuario*> _usuarios; // vector de usuarios
58     vector<Evento*> _eventos; // vector de eventos
59     vector<Localizacion*> _localizaciones; // vector de eventos
60 };

```

Por último hemos usado otro vector en la clase asistente el cual contiene las entradas que tiene cada asistente.

```

43     vector <Entrada*> _entradas;

```

3.7 Sistema de compra de entradas y su gestión:

Primero hemos creado en asistente una función la cual añade entradas al vector entradas que hemos mencionado anteriormente y otra que las muestra por pantalla:

```

90 void Asistente::anadirEntrada(Entrada* entrada)
91 {
92     _entradas.push_back(entrada);
93     cout << "Entrada añadida a tu lista." << endl;
94 }
95
96 void Asistente::mostrarEntradas() const
97 {
98     if (_entradas.empty()) //si el vector esta vacio
99     {
100         cout << "No tienes entradas compradas." << endl;
101         return;
102     }
103
104     cout << "- Tus entradas: " << endl;
105     for (int i = 0; i < _entradas.size(); ++i) {
106         cout << i + 1 << ". ";
107         _entradas[i]->display(); //muestra cada entrada
108     }
109 }

```

Las cuales nos serán útiles para la gestión de las entradas que realizamos en la clase interfaz, donde mostramos por pantalla una lista de todos los eventos disponibles y dependiendo de si tenemos fondos suficientes en la cartera y si eres VIP o no podrás comprar la entrada

```
439 void Interfaz::comprarEntrada(Asistente* asistente)
440 {
441     mostrarEventos();
442
443     cout << "Selecciona el número del evento que desea comprar: ";
444     int seleccion;
445     cin >> seleccion;
446
447     if (seleccion < 1 || seleccion > _eventos.size()) //si no esta entre los numeros nos salimos
448     {
449         cout << "Evento no válido." << endl;
450         return;
451     }
452
453     Evento* evento = _eventos[seleccion - 1]; //seleccionamos el evento - 1 porque los mostramos como i + 1
454
455     if (evento->getvip() && !dynamic_cast<AsistenteVIP*>(asistente)) //si es un evento solo para vips
456     {
457         cout << "Este evento es VIP. Solo los asistentes VIP pueden comprarlo." << endl;
458         return;
459     }
460
461     float precio = evento->getprecio();
462
463     if (asistente->getCartera() < precio) //si no lo puede comprar
464     {
465         cout << "Fondos insuficientes para realizar la compra." << endl;
466         return;
467     }
468
469     asistente->setCartera(asistente->getCartera() - precio); //restamos a su cartera el precio
470     asistente->anadirEntrada(new Entrada(evento, precio, asistente)); // Añade la entrada al asistente
471
472     cout << "Entrada comprada con éxito." << endl;
473
474 }
```

Esta función la llamamos en el main, en el menú de los asistentes, los cuales pueden comprar funciones.

3.8 Almacenamiento en ficheros:

Para el almacenamiento en ficheros, hemos creado tres ficheros. Uno para usuarios en el que guardamos toda clase de usuarios, otro para eventos y el último para localizaciones.

Para todos los ficheros, antes de crear la función en la clase interfaz, hemos creado una función en cada clase que vamos a guardar en ficheros que imprime cada atributo de la clase.

```
89 void Artista::guardarEnFichero(ostream& os) const
90 {
91     os << "-Tipo: Artista " << endl;
92     os << " -Nombre de usuario: " << getnombreUsuario() << endl;
93     os << " -Contraseña: " << getcontrasena() << endl;
94     os << " -Nombre: " << getnombre() << endl;
95     os << " -Estilo: " << getEstilo() << endl;
96     os << " -Descripcion: " << getDescripcion() << endl;
97 }
```

Y luego una función en la clase interfaz que recorre cada vector e imprime en el fichero de la manera que hemos creado en la función de cada clase.

```
306 void Interfaz::guardarUsuariosEnFichero() const
307 {
308     ofstream fichero("usuarios.txt");
309
310     if (!fichero.is_open())
311     {
312         cout << "Error al abrir el fichero" << endl;
313         return;
314     }
315
316     for (int i = 0; i < _usuarios.size(); i++)
317     {
318         fichero << i+1 << ". ";
319         _usuarios[i]->guardarEnFichero(fichero);
320         fichero << "======" << endl; // Separador entre usuarios
321     }
322
323     fichero.close();
324     cout << "Fichero de Usuarios creado correctamente." << endl;
325 }
```

Estas funciones las llamamos al final del main para que imprima en los ficheros tras la ejecución del programa. Quedandonos así tres ficheros, "usuarios.txt", "eventos.txt" y "localizaciones.txt"

usuarios.txt	eventos.txt	localizaciones.txt
1. -Tipo: Administrador -Nombre de usuario: admin1 -Contraseña: adminpass1 =====	1. Evento -Artista: Shakira -Localización: WiZink Center -Fecha: 01/07/2024 -Precio: 10 € -Evento VIP: No -Entradas disponibles: 5000 =====	1. Localizacion -Nombre: WiZink Center -Direccion: Av. Felipe II, Madrid -Aforo: 17000 =====
2. -Tipo: Administrador -Nombre de usuario: admin2 -Contraseña: adminpass2 =====	2. Evento -Artista: Metallica -Localización: Sala Riviera -Fecha: 10/07/2024 -Precio: 5 € -Evento VIP: Sí -Entradas disponibles: 2500 =====	2. Localizacion -Nombre: Sala Riviera -Direccion: Paseo Bajo, Madrid -Aforo: 2500 =====
3. -Tipo: Administrador -Nombre de usuario: admin3 -Contraseña: adminpass3 =====	3. Evento -Artista: Bad Bunny -Localización: Palau Sant Jordi -Fecha: 15/07/2024 -Precio: 3.4 € -Evento VIP: No -Entradas disponibles: 10000 =====	3. Localizacion -Nombre: Palau Sant Jordi -Direccion: Barcelona -Aforo: 18000 =====
4. -Tipo: Asistente -Nombre de usuario: josema123 -Contraseña: 123jm -DNI: 12345678A -Cartera: 30.02 € =====		4. Localizacion -Nombre: Estadio La Cartuja -Direccion: Sevilla -Aforo: 60000 =====
5. -Tipo: Asistente -Nombre de usuario: jorgep456 -Contraseña: 456jp -DNI: 12345678B -Cartera: 55.7 € =====		5. Localizacion -Nombre: Teatro Real -Direccion: Madrid -Aforo: 1750 =====

Esta es la información que guardamos en los ficheros.

3.9 Menús e interfaz:

Hemos creado una clase menú la cual contiene todos los menús que necesitamos:

```

7 void Menu::menuinicio()
8 {
9     cout << "==== PAGINA DE INICIO =====< endl;
10    cout << "1. Registrarse" << endl;
11    cout << "2. Iniciar sesión" << endl;
12    cout << "3. Salir del programa" << endl;
13    cout << "Seleccione una opcion: ";
14 }
15 void Menu::menuadministrador()
16 {
17     cout << "==== MENU DE ADMINISTRADORES =====< endl;
18     cout << "1. Crear usuario" << endl;
19     cout << "2. Modificar usuario" << endl;
20     cout << "3. Eliminar usuario" << endl;
21     cout << "4. Mostrar usuarios" << endl;
22     cout << "5. Mostrar eventos" << endl;
23     cout << "6. Mostrar localizaciones" << endl;
24     cout << "7. Salir del programa" << endl;
25     cout << "Seleccione una opcion: ";
26 }
27 void Menu::menuartista()
28 {
29     cout << "==== MENU DE ARTISTAS =====< endl;
30     cout << "1. Crear evento" << endl;
31     cout << "2. Modificar evento" << endl;
32     cout << "3. Eliminar evento" << endl;
33     cout << "4. Mostrar localizaciones" << endl;
34     cout << "5. Salir del programa" << endl;
35     cout << "Seleccione una opcion: ";
36 }

```

```

37 void Menu::menuasistente()
38 {
39     cout << "==== MENU DE ASISTENTES =====< endl;
40     cout << "1. Comprar entradas" << endl;
41     cout << "2. Mostrar eventos" << endl;
42     cout << "3. Mostrar tus entradas" << endl;
43     cout << "4. Salir del programa" << endl;
44     cout << "Seleccione una opcion: ";
45 }
46 void Menu::menuasistentevip()
47 {
48     cout << "==== MENU DE ASISTENTES VIP =====< endl;
49     cout << "1. Comprar entradas" << endl;
50     cout << "2. Mostrar eventos" << endl;
51     cout << "3. Mostrar tus entradas" << endl;
52     cout << "4. Salir del programa" << endl;
53     cout << "Seleccione una opcion: ";
54 }

```

Estos menús los manejamos con switches en el main donde vamos eligiendo a qué opción realizar.

En el main, primero mostramos el menu de inicio en el que tenemos la opción de registrarse o iniciar sesión

```

78     case 1:
79         interfaz.crearUsuario(); //creamos un usuario
80         usuarioActivo = interfaz.getUsuarios().back(); // como usamos psuhback con el .back devuelve el ultimo
81         cout << endl << "Registro completado, " << usuarioActivo->getnombreUsuario() << endl;
82         break;
83     case 2:
84     {
85         string nombreUsuario, contrasena;
86
87         int intentos = 0, numintentos=3;
88
89         while (intentos < numintentos && usuarioActivo == nullptr) //mientras no hagas mas de 3 intentos y el us
90         {
91             cout << "Nombre de usuario: ";
92             cin >> nombreUsuario;
93             cout << "Contraseña: ";
94             cin >> contrasena;
95
96             for (int i = 0; i < interfaz.getUsuarios().size(); i++) // recorre la lista buscando al usuario
97             {
98                 Usuario* u = interfaz.getUsuarios()[i]; //usuario auxiliar
99                 if (u->getnombreUsuario() == nombreUsuario && u->getcontrasena() == contrasena) //comprobacion
100                 {
101                     usuarioActivo = u;
102                     break; // Si lo conseguimos salimos sin recorrer mas
103                 }
104             }
105
106             if (usuarioActivo) //si el usuario activo es verdadero
107             {
108                 cout << "Inicio de sesion completado, " << usuarioActivo->getnombreUsuario() << endl;
109             }
110             else
111             {
112                 cout << "Usuario o contraseña incorrectos. Inténtalo de nuevo.\n" << endl;
113                 intentos++;
114             }
115         }
116
117         if (!usuarioActivo) //si despues de los tres intentos el usuario activo es falso
118         {
119             cout << "Demasiados intentos. " << endl;
120             return 0; //se termina el programa
121         }
122     }
123     break;

```

Ejemplo de registro:

```
===== PAGINA DE INICIO =====
1. Registrarse
2. Iniciar sesión
3. Salir del programa
Seleccione una opcion: 1
Introduce nombre de usuario: rafat
Introduce contraseña: 123456
Tipo de usuario (1. Artista, 2. Asistente, 3. AsistenteVIP, 4. Administrador): 1
Introduce nombre: Rafa Music
Introduce estilo musical: Pop
Introduce descripcion: Cantante de pop joven
Usuario creado correctamente.

Registro completado, rafat
===== MENU DE ARTISTAS =====
1. Crear evento
2. Modificar evento
3. Eliminar evento
4. Mostrar localizaciones
5. Salir del programa
Seleccione una opcion: █
```

Ejemplo de inicio de sesión:

```
===== PAGINA DE INICIO =====
1. Registrarse
2. Iniciar sesión
3. Salir del programa
Seleccione una opcion: 2
Nombre de usuario: admin1
Contraseña: adminpass123
Usuario o contraseña incorrectos. Inténtalo de nuevo.

Nombre de usuario: admin1
Contraseña: adminpass1
Inicio de sesion completado, admin1
===== MENU DE ADMINISTRADORES =====
1. Crear usuario
2. Modificar usuario
3. Eliminar usuario
4. Mostrar usuarios
5. Mostrar eventos
6. Mostrar localizaciones
7. Salir del programa
Seleccione una opcion: █
```

Tras el inicio de sesión o el registro, dependiendo del tipo de usuario que hayas elegido se marchara a un menú u otro, esto es gracias al `dinamica_cast`, el cual identifica el tipo de usuario que eres y te manda a un menú u otro.

3.10 Sobrecarga de operadores:

Por último hemos implementado la sobrecarga de los operadores `<<` y `>>` para las clases de usuario y para eventos, lo que nos ha facilitado que podamos escribir e imprimir estos objetos.

```

45  ostream& operator<<(ostream& os, const Artista& artista)
46  {
47      os << "-Tipo: Artista " << endl;
48      os << " -Nombre de usuario: " << artista.getnombreUsuario() << endl;
49      os << " -Contraseña: " << artista.getcontrasena() << endl;
50      os << " -Nombre: " << artista.getnombre() << endl;
51      os << " -Estilo: " << artista.getEstilo() << endl;
52      os << " -Descripcion: " << artista.getDescripcion() << endl;
53      os << " ===== " << endl;
54
55      return os;

```

```

57  istream& operator>>(istream& is, Artista& artista)
58  {
59      string nombreusuario, contrasena, nombre, estilo, descripcion;
60
61      cout << "Introduce los datos del Artista:" << endl;
62
63      cout << "Nombre de usuario: ";
64      is >> nombreusuario;
65      artista.setnombreUsuario(nombreusuario);
66
67      cout << "Contraseña: ";
68      is >> contrasena;
69      artista.setcontrasena(contrasena);
70
71      //cuando hay espacios en cin da problemas asi que usaremos el getline
72      cout << "Nombre: ";
73      is.ignore(); // Limpia el salto de línea pendiente
74      getline(is, nombre);
75      artista.setnombre(nombre);
76
77      cout << "Estilo: ";
78      is.ignore(); // Limpia el salto de línea pendiente
79      getline(is, estilo);
80      artista.setEstilo(estilo);
81
82      cout << "Descripcion: ";
83      is.ignore(); // Limpia el salto de línea pendiente
84      getline(is, descripcion);
85      artista.setDescripcion(descripcion);
86
87      return is;
88  }

```

4. Optimización y conclusión.

Como conclusión, tras haber escrito muchas líneas de código, las principales complicaciones han sido el tema del desorden, por eso decidimos tomarnos unos días simplemente para crear clases nuevas con la finalidad de tener todo algo más compartimentado, como es el caso de interfaz la cual maneja todo el programa dejando así un código en el main muy limpio y claro. Otra cosa que nos ayudó mucho a optimizar fue el polimorfismo en los usuarios, al tener todos los tipos de

usuarios en el mismo vector, solo necesitábamos crear una función genérica para todos los usuarios y estos hacían sus funciones.

El trabajo nos ha gustado mucho, aunque ha sido extenso, lo hemos intentado llevar al día. Nos hubiese gustado implementar algún objetivo máximo pero estamos contentos con el resultado.